



UNIVERSITY OF FRONTIER TECHNOLOGY, BANGLADESH

Department of Cyber Security Engineering

Project Proposal

Name of the project : Zero Trust Access Control System

Course Name : Object Oriented Programming Language Sessional

Course Code : PROG 112

Instructor :

Name	Designation	Department
Md Masud Rana	Lecturer	Cyber Security Engineering

Team Members :

Name	Student ID	level	Term
Mst. Bithi Khatun	2404024	01	02
Afra Anjum Odhora	2404025	01	02

Zero Trust Access Control System

Project Overview

The Zero Trust Access Control System is a secure, console-based access management prototype that implements the core principles of Zero Trust Architecture — “Never Trust, Always Verify”.

Unlike traditional security models that trust users after initial login, this system continuously verifies every access request based on user identity, session validity, role-based permissions, and risk factors.

The project simulates a real-world organizational access environment (similar to banking or enterprise systems) using C++ and Object-Oriented Programming (OOP). It includes Multi-Factor Authentication, Role-Based Access Control, Session Management, Re-authentication, Suspicious Activity Detection, and Encrypted Audit Logging.

Problem Statement

Bangladesh is undergoing rapid digital transformation through mobile financial services (bkash, Nagad, Rocket), internet banking, e-governance, and online education platforms. However, this growth has significantly increased cybersecurity threats.

Key challenges include:

- The financial sector faces 400630 cyberattacks per day.
- The 2016 Bangladesh Bank heist (USD 81 million stolen) highlighted the severe consequences of weak access control and verification systems.
- Most organizations still rely on traditional perimeter-based security systems (simple username/password), which is highly vulnerable to credential stuffing, phishing, and lateral movement attacks.
- Limited budget allocation for cybersecurity and lack of skilled professionals.
- Bangladesh Bank’s Cybersecurity Framework 2026 emphasizes the urgent need for stronger access control and continuous monitoring.

Traditional “trust once” models are no longer sufficient. There is a critical need for practical demonstration and understanding of Zero Trust Architecture in the Bangladeshi context.

Proposed Solution & Objectives

Proposed Solution:

Develop a fully functional console-based Zero Trust Access Control System in C++ that enforces continuous verification at every step of user interaction.

Objectives:

- Implement strong Multi-Factor Authentication (MFA) to prevent unauthorized access.
- Enforce Role-Based Access Control (RBAC) for different user privileges.
- Demonstrate the “Never Trust, Always Verify” principle through continuous verification.
- Simulate realistic session management with automatic logout on inactivity.
- Implement re-authentication for sensitive actions.
- Detect and alert suspicious activities in real-time.
- Provide secure encrypted audit logging.
- Create an interactive demo to showcase various security scenarios.

Target Audience

Cyber Security Students

- Understand authentication, authorization, and session management.

Academic Instructors

- Use the system as a teaching tool for cybersecurity and secure coding.

Beginners in Secure System Design

- Learn how to design a secure authentication system.
- Understand access control mechanisms like Role-Based Access Control (RBAC).

Small to Medium Enterprises / Institutions

- Understand and explore Zero Trust security principles before real-world implementation.
- Use the system as a training and awareness tool for employees.

Key Features & Project Scope

The project implements the following key features:

1. Multi-Factor Authentication (MFA)

- Uses both password and OTP verification to ensure stronger user authentication.
- Reduces the risk of unauthorized access.

2. Login Attempt Limit

- Limits login attempts to a maximum of 3 tries to prevent brute-force attacks.
- Automatically locks the account temporarily after repeated failed attempts.

3. Role-Based Access Control (RBAC)

- Assigns users different roles (Admin, User, Guest) with specific permissions.
- Ensures users can only access resources allowed for their role.

4. Continuous Verification

- Verifies user identity and session on every access request.

5. Session Management

- Starts a session after login and automatically expires after inactivity.
- Requires re-authentication if the session is no longer valid.

6. Re-authentication System

- Requests the user to re-enter the password for sensitive actions.
- Determines whether to Allow, Deny, or Re-authenticate each request.

7. Access Decision Engine

- Decides Allow / Deny / Re-authenticate

8. Suspicious Activity Detection

- Detects unusual behavior such as repeated denied access attempts.
- Generates alerts to indicate potential security threats.

9. Encrypted Logging System

- Stores all system activities in an encrypted log file (audit.log).
- Protects sensitive log data from being easily read or tampered with.

Functional & Non-Functional Requirements

Functional Requirements

- Users can login using MFA (Password + OTP).
- System locks account after 3 failed attempts.
- Role-specific access rights enforced.
- Every request passes through Access Decision Engine.
- Session expires after inactivity and requires re-authentication.
- Sensitive actions trigger re-authentication.
- Suspicious activities are detected and alerted.
- All events are encrypted and logged.

Non-functional requirements

1. Security

- Strong MFA, continuous verification, encrypted logging.

2. Performance

- Responsive interface with thread-based session monitoring.

3. Reliability

- Proper exception handling and input validation.

4. Maintainability

- Clean modular OOP design.

5. Portability

- Runs on Code::Blocks (Windows/Linux).

Technology Stack

- **Programming Language:** C++.
- **IDE:** Code::Blocks.
- **Libraries:** Standard Template Library (STL), <thread>, <mutex>, <fstream>
- **Design Approach:** Object-Oriented Programming with Design Patterns.

System Architecture & UI Design

System Architecture

The architecture follows a six-layer architecture:

1. Presentation Layer

- ❖ Menu-driven Console User Interface.
- ❖ Handles all user interactions and displays colored outputs.

2. Authentication Layer

- ❖ Responsible for Multi-Factor Authentication (Password + 4-digit OTP).
- ❖ Manages Login Attempt Limit and Account Lock mechanism.
- ❖ Includes Re-authentication module for sensitive actions.

3. Session Management Layer

- ❖ Creates and tracks user sessions.
- ❖ Implements thread-based inactivity timer for automatic session expiration.
- ❖ Enforces re-authentication when session expires.

4. Authorization & Decision Layer (Core of Zero Trust)

- ❖ Access Decision Engine – The brain of the system.
- ❖ Performs Continuous Verification on every request.

5. Security & Monitoring Layer

- ❖ Suspicious Activity Detection module.
- ❖ Risk assessment based on failed attempts and denied accesses.
- ❖ Triggers real-time security alerts.

6. Logging & Auditing Layer

- ❖ Secure Encrypted Logging System (audit.log).
- ❖ Uses XOR encryption technique for data protection.

UI Design

The user interface will include:

- Role-based dashboards.
- Successful MFA login.
- Account Lock after 3 failed attempts.
- Session timeout & Re-authentication.
- Role-based access restriction.
- Suspicious activity detection.

Project Timeline & Milestones

Milestone No.	Activities	Duration
1	User Hierarchy and RBAC Implementation	Week 1
2	MFA login and Attempt Limit	Week 2
3	Session Management with thread	Week 3
4	Access Decision Engine and Re-authentication	Week 4
5	Suspicious Detection and Encrypted Logging	Week 5
6	Menu UI and Demo Scenarios and Testing	Week 6
7	Documentation	Week 7

Expected Deliverables

- Complete working C++ source code.
- Encrypted audit.log file with decryption utility.
- Full Project Report & Documentation.
- README.md with setup and usage instructions.
- Class Diagram and System Architecture Diagram.